

Gestionnaire de Suivi d'Expériences Professionnelles

BE-Flutter est une application bureau développée avec **Flutter**, conçue pour permettre de suivre, noter et analyser leurs journées de travail au jour le jour.

Initialement inspirée d'une version Qt/C++, cette réécriture moderne tire parti de l'écosystème Flutter pour offrir une interface fluide et réactive sur Windows (et potentiellement mobile).

Fonctionnalités

1. Gestion des Expériences

- **Création** : Ajoutez de nouvelles périodes professionnelles (titre, dates).
- **Modification** : Éditez les informations d'une expérience existante.
- **Suppression** : Retirez les expériences obsolètes.
- **Liste globale** : Vue d'ensemble de toutes vos missions passées et actuelles.

2. Suivi Quotidien (Journaling)

Pour chaque expérience, vous pouvez enregistrer des entrées journalières :

- **Date** : Sélection précise via calendrier.
- **Notation** : Évaluation de la journée sur une échelle de 1 à 5.
- **Respect du contrat** : Indicateur binaire (Oui/Non) pour suivre la conformité des missions.
- **Calcul automatique** : Moyenne des notes mise à jour en temps réel.

3. Visualisation et Analyse

- **Détails** : Vue approfondie de chaque expérience.
- **Statistiques** : Affichage des moyennes et des indicateurs de performance.
- **Graphiques** : Zone dédiée à la visualisation de la progression (*En cours d'implémentation*).

4. Persistance des Données

- Sauvegarde locale automatique pour conserver vos données entre les sessions.

Stack Technique

Le projet repose sur une architecture robuste et des librairies éprouvées :

Catégorie	Technologie	Usage
Framework	Flutter	UI Multiplateforme (Windows focus)
Langage	Dart 3	Logique métier

Catégorie	Technologie	Usage
State Management	Riverpod	Gestion d'état réactive et injection de dépendances
Persistence	Shared Preferences	Stockage local simple
Sérialisation	Json Serializable	Conversion automatique JSON <-> Objets Dart
Code Gen	Build Runner	Génération de code (pour les modèles)
Internationalisation	Intl	Formatage des dates

Architecture du Projet

Le code source est organisé selon une structure modulaire séparant clairement les responsabilités :

```
lib/
├── models/           # Définition des entités (Day, Experience) + fichiers
                        .g.dart générés
├── pages/            # Écrans principaux de l'application (Home, Details)
├── providers/        # Logique d'état (ExperienceProvider) - Lien entre UI et
Data
├── repositories/     # Couche d'accès aux données (Abstraction du stockage)
├── widgets/          # Composants UI réutilisables
│   ├── experience_details/ # Composants spécifiques à la vue détail
│   └── home/          # Composants spécifiques à l'accueil
└── main.dart         # Point d'entrée de l'application
```

Guide d'Installation

Prérequis

- **Flutter SDK** (version stable recommandée)
- **Dart SDK**
- Un IDE (VS Code, Android Studio, ou IntelliJ)

Étapes

1. Cloner le projet

```
git clone https://github.com/votre-username/BE-Flutter.git
cd BE-Flutter
```

2. Installer les dépendances

```
flutter pub get
```

3. Générer les fichiers de modèle (Indispensable)

Le projet utilise la génération de code pour le JSON. Vous devez lancer cette commande avant le premier build ou après avoir modifié les modèles.

```
dart run build_runner build --delete-conflicting-outputs
```

4. Lancer l'application

```
flutter run -d windows
```

Développement

Commandes utiles

- **Lancer les tests :**

```
flutter test
```

- **Régénérer le code en continu (Watch mode) :**

Si vous travaillez sur les modèles (`models/`), utilisez cette commande pour régénérer automatiquement les fichiers `.g.dart` à chaque sauvegarde.

```
dart run build_runner watch --delete-conflicting-outputs
```

Auteur

Projet personnel de conversion et d'amélioration d'un outil de productivité.